Base: An Interface for Programming the Board

https://github.com/unnamedlambda/base

Abstract

Base is an interface and system for programming at the scope of an entire board. The CPU, GPU, display and file system are all controlled through a single artifact loaded by a driver. Its compiled entry points can then be called at run time, with the driver embedded through its C API and usable from Rust, Python, C and Lean. Lean generators for artifacts have theorems which prove, at build time, properties of the run-time behavior. Performance matches standalone Rust on CPU workloads and PyTorch on GPU workloads.

1 System Architecture

Modern computer applications run on many devices at once, each with its own control interface. A CPU has an instruction set, which a language like C is compiled to directly. A GPU has a driver API, reached through system software and usually programmed in a kernel language of its own such as CUDA [3]. Displays and file systems are reached by system calls the CPU makes, with values computed at run time. Each of these interfaces is precise in isolation, but real applications require them to be composed together correctly. The programmer must reason about CPU arithmetic, GPU kernel launches and their results, file paths and the contents of writes, and above all about the interleaving of data between these interfaces across the program's whole run.

For a single general-purpose device this is solved by an instruction set: a finite, documented vocabulary through which the device's components are controlled. A program for that instruction set is a standalone object, independent of source details. Whether it was written by hand or generated by a compiler, the same final object is the whole of the interface and governs the resulting behavior. Base extends the scope of such an interface to the entire board. A software driver accepts full board control via an *artifact*: a value that can be serialized, exist in memory, and be the subject of logical reasoning.

1.1 Artifact

An artifact is represented as the Rust struct below and is encoded as the CBOR data format [4]. It contains functions, the primitives they call, and the memory they start with.

```
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
#[serde(deny_unknown_fields)]
pub struct Artifact {
    pub primitives: Vec<clif::Primitive>,
    pub functions: Vec<clif::Function>,
    pub arena_size: u64,
    pub data: Vec<Segment>,
}
```

A user loads a serialized artifact, looks up an entry point by name, and calls it.

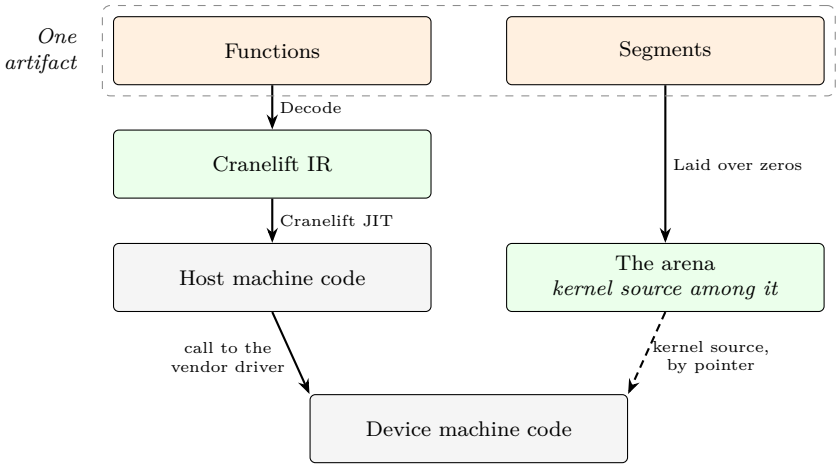


Figure 1: The functions are decoded and compiled for the host processor. The segments are laid into the arena, kernel source among them, and that source reaches the vendor's driver.

```
let driver = unsafe { Driver::load(ARTIFACT_BYTES)? };
let infer = driver.function("infer")?;

let status = infer(&input, &mut output);
```

The functions are CLIF, the IR of the Cranelift code generator [1]. During load the driver JIT compiles the artifact's functions to machine code and initializes a shared arena according to the artifact's specification. `arena_size` is how much space to reserve for the arena, and the segments of `data` are laid over it, so an artifact isn't forced to specify every initial zero. A function is looked up by a name the artifact carries, and called with the caller supplying both the input and output buffers. The same arena is shared between calls, so state carries across them: a device handle opened by one entry point is still open for the next.

1.2 Driver

Each CLIF function carries an optional name by which a host looks it up. A function's blocks, block parameters and instructions are encoded as data rather than as raw text. The driver walks that structure through Cranelift's instruction builder to produce machine code. The generators use a subset of CLIF: eight types, the widest a 128-bit vector, and eighty-six opcodes spanning arithmetic, comparison, bit manipulation, conversion, memory access, calls and structured control flow.

CLIF functions control one core and the memory it addresses, and reach for *primitives* to access the rest of the board. A primitive is a C function compiled ahead of time. When an artifact is loaded, the driver hands Cranelift each primitive's address to enable direct calls into these functions.

Device kernels can be embedded into the artifact as source text in its data segments, and are run through the vendor's own API. The `cuModuleLoadData` primitive takes kernel source out of the arena and gives it to the vendor's compiler; `cuLaunchKernel` launches the result, and `cuMemcpyDtoH` writes what it computed back into the arena. Figure 1 follows both paths out of the artifact. Kernel source and vendor driver use are handled entirely this way; Base has no built-in device handling outside what a program reaches through these primitives.

Figure 2 shows how CLIF and the primitives control board behavior. An exploration of a specific primitive, for reading a file into a buffer, follows.

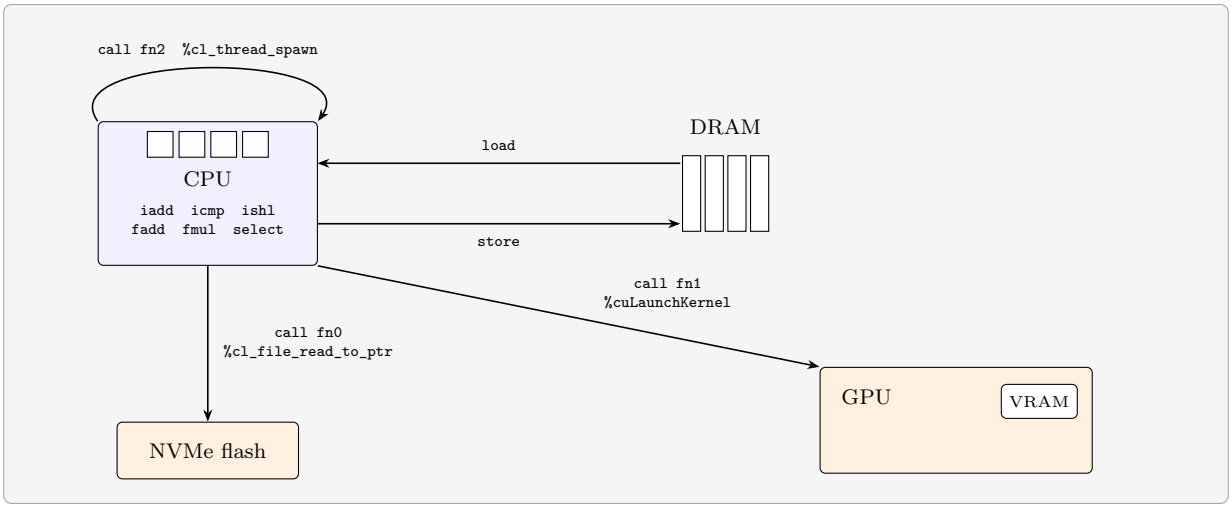


Figure 2: A system board along with how its capabilities are accessed by CLIF. Arithmetic and memory access are instructions such as `iadd`, loads and stores. Everything else is reached by a call to a board primitive.

```
sig0 = (i64, i64, i64, i64, i64) -> i64 system_v
fn0 = %cl_file_read_to_ptr sig0

v6 = call fn0(v1, v2, v3, v4, v5)
```

Figure 3: Cranelift makes a call to `cl_file_read_to_ptr` just as it would any other function. The CLIF text format is shown for clarity.

```
pub(crate) unsafe extern "C" fn cl_file_read_to_ptr(
    path_ptr: *const u8,
    path_cap: i64,
    dst_ptr: *mut u8,
    file_offset: i64,
    size: i64,
) -> i64 { ... }
```

It takes a file path and a destination to write the file contents to, and returns the count read, or -1 to indicate a failure. The pointers are plain addresses; for a CLIF program they point into the arena, the only memory it has. The call itself is unremarkable: it sits among the arithmetic that computed its arguments, and what it returns is a value like any other (Figure 3).

1.3 C API

The entire Base driver is exported as a C API, shown below. This enables the driver to be embedded for calling from other languages, as C is the common foreign function interface. C++, Python and Lean APIs are built on top of this.

```
#[no_mangle]
pub unsafe extern "C" fn base_driver_load(
    artifact: *const u8, len: usize,
    err: *mut u8, err_cap: usize) -> *mut Driver { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_function(
    driver: *const Driver,
    name: *const u8, name_len: usize) -> Option<BaseFunction> { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_arena(
    driver: *const Driver, len: *mut usize) -> *mut u8 { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_free(driver: *mut Driver) { ... }
```

2 Programming Experience

The driver's capabilities are accessed through the artifact, so it is clear why the construction of an artifact must be done carefully. CLIF's instructions and control-flow features have documented behavior, which enables reasoning about correctness.

2.1 Artifact Generation

Artifacts are produced programmatically, and a system that does this is called a *generator*. A generator relates to an artifact much as a C compiler relates to its object code. Figure 4 puts this relation side by side.

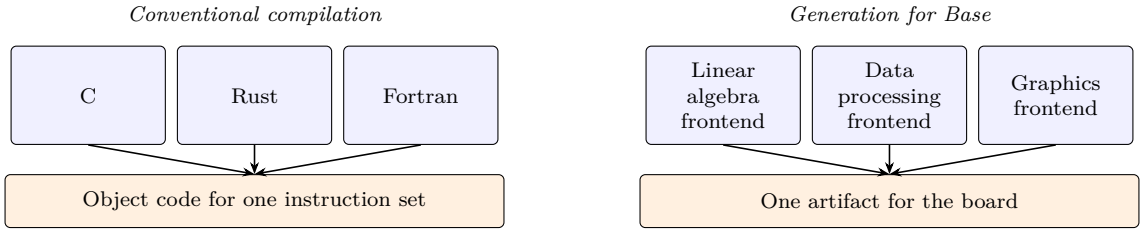


Figure 4: On the left source languages are compiled into native instructions for the processor being targeted. On the right Base enables generation of artifacts from different frontends.

Lean [7] serves as the interface from which artifacts are generated. Any language or proof assistant which is capable of accessing Base through the C API is a valid candidate for a generator, but no alternative has been implemented. Artifact contents are Lean terms, serialized into CBOR and used with the driver interface.

The program below replaces every NUL byte in a buffer with a space. Lean's `do` notation over a `Prog` term binds values much as an IR builder does in an application with an embedded compiler. The Lean here runs to produce the artifact value, which is what performs the byte replacement.

```
def blend (v nul space : V .i8x16) : Prog V L (V .i8x16) := do
  bitselect (← icmp .eq v nul) space v

def scrub (vectors : Nat) (src dst : V .i64) : Prog V L Unit := do
  let nul ← splat .i8x16 (← iconst .i8 0)
  let space ← splat .i8x16 (← iconst .i8 32)
  forLoop (← iconst64 vectors) fun i => do
    let off ← ishlImm i 4
    let v ← loadI8x16 (← iadd src off)
    store (← blend v nul space) (← iadd dst off)

def code : Prog V L Unit := do
  let src ← dataPtr
  let dst ← outPtr
  let need ← iconst64 4096
  when .ule need (← dataLen) do
    when .ule need (← outLen) do
      scrub 256 src dst
```

Figure 5 follows three lines of the scrub loop from Lean through CLIF to the machine code Cranelift emits.

Lean, by the user	*CLIF, in the artifact*	*x86-64, for the host*
`loadI8x16 (← iadd src off)`	`v16 = iadd.i64 v1, v15` `v17 = load.i8x16 notrap aligned v16`	`vmovdqu (%rsi, %rdi), %xmm2`
`bitselect (← icmp .eq v nul)` `  space v`	`v18 = icmp eq v17, v8` `v19 = bitselect.i8x16 v18, v10, v17`	`vpcmpeqb %xmm3, %xmm2, %xmm3` `vpblendvb %xmm3, %xmm4, %xmm2, %xmm2`
`store (← blend v nul space)` `  (← iadd dst off)`	`v20 = iadd.i64 v3, v15` `store notrap aligned v19, v20`	`vmovdqu %xmm2, (%rcx, %rdi)`

Figure 5: Three representations of the same computation as handled by Base.

2.2 Reasoning About Emitted Code

The intention of the scrub code above is to leave no NUL bytes in the buffer. Below is a statement about the emitted instructions which are generated for the artifact. Each input byte is confirmed to carry through to the output, with the desired substitution performed.

```
def Answers (Γ : Env) (c : Code) (binds : Nat) (v : V) : Prop :=
  ∀ (fuel : Nat) (cfg : Cfg) (w : World), ∃ Γ',
    runCode (fuel + 2) cfg Γ w c = .ok Γ' w
    ∧ Γ'.size = Γ.size + binds
    ∧ Γ'.back? = some v

theorem blend_leaves_no_nul
    (Γ : Env) (tys : List ClifTy) (hΓ : tys.length = Γ.size)
    (a b c : R) (xs : Array UInt64) (h16 : xs.size = 16)
    (ha : Γ[a]? = some (.vec .i8x16 xs))
    (hb : Γ[b]? = some (.vec .i8x16 (.replicate 16 0)))
    (hc : Γ[c]? = some (.vec .i8x16 (.replicate 16 32))) :
    ∃ ys, ys = xs.map (fun x => if x = 0 then 32 else x)
          ∧ Answers Γ (Prog.emitTerm (discard (blend a b c)) tys) 2 (.vec .i8x16 ys)
          ∧ ∀ y ∈ ys, y ≠ 0 :=
  ⟨scrubLanes xs, rfl,
   fun fuel cfg w =>
     ⟨_, blend_scrubs fuel cfg w Γ tys hΓ a b c xs h16 ha hb hc, by simp,
      by simp [scrubbedBytes]⟩,
   scrubLanes_no_nul xs⟩
```

The theorem above does not cover the splats or any behavior of the loop around the blend; however, similar theorems could be added to cover this and the remaining behavior.

2.3 Accessing the Board

A program that drives other device capabilities besides the CPU is written with the same Lean interface. The following example renders a 4096×4096 Mandelbrot set on the GPU and writes it to a bitmap file. The artifact's data segments hold the GPU kernel source (the shader) and other values used at run time.

```
def code : Prog V L Unit := do
  let ptr ← basePtr
  app.setup ptr
  app.dispatch ptr 0 (← iconst32 wgX) (← iconst32 wgY) (← iconst32 1)
  app.download ptr 0 (← iaddImm ptr f.pixels.offset) (← iconst64 pixelBytes)
  app.cleanup ptr
  let total  ← iconst64 (54 + pixelBytes)
  let _      ← fldWriteFile0 ptr f.filename f.bmpHeader total
```

3 Evaluation

Base has its speed evaluated on workloads to ensure it can maintain the speeds that are achievable with systems languages. The comparison is made with standalone Rust [5] compiled through LLVM [2] implementing the same workload. To assess device work, GPU workloads in Base are compared with PyTorch [6].

Table 1: A comparison for workloads written in idiomatic low-level Rust, Base using only CLIF, and Base generating native assembly, on an AMD Ryzen 5 5500.

Workload	Rust (μs)	CLIF / Rust	asm / Rust	Cause
Mandelbrot set	1800.6	1.00		
Pointer chase	2297.9	1.00		
Byte histogram	320.0	1.00		
Polynomial, `f32`	6.1	2.02	1.01	128-bit vectors
Copy, 32 MB	2288.4	1.58	1.00	no non-temporal store

3.1 CPU Workloads

Base with CLIF is level with Rust for the first three workloads. The other two see Rust accessing capabilities that CLIF lacks support for: 256-bit vectors for the polynomial and non-temporal stores for the copy. To close this gap, Base uses native assembly, assembled in Lean and accessed using the `cl_native_load` primitive.

3.2 GPU Workloads

On GPU workloads Base is comparable to PyTorch in whichever of eager and compiled mode is faster. Small workloads are faster in Base, where fusion and a lower cost of dispatch improve over PyTorch.

Table 2: GPU workload comparisons on an RTX 3060.

Workload	PyTorch (μs)	Base / PyTorch
Vector add 50M	1810	1.00
Scale-and-add 50M	1850	0.98
Scale-and-add 1M	84	0.60
GEMV 11008×4096	554	0.98
RMSNorm 4096	55	0.20
Softmax 32000	31	0.35
Decode attention, 2048 keys	83	0.77
Decode attention, 128 keys	66	0.24

4 Limitations

While Lean enables mechanical reasoning over the artifact and the code used to generate it, the components of the infrastructure that it cannot cover are worth considering. The model of CLIF is translated from the documentation and tested against Cranelift on a corpus of 1764 cases. The driver and the primitives are implemented and tested in Rust or supplied by the vendors, and no Lean proof can reach into these implementations; it can only state assumptions about them, which can be reviewed by hand.

Not every application can accommodate embedding the Base driver along with the primitive implementations. Base is portable across x86-64 and ARM and uses no features specific to one operating system, but platforms outside these are not a fit, nor are applications that cannot accept the driver's cost at load: the most expensive artifact in the repository takes 27 ms to decode and 140 ms to compile.

5 Conclusion

Base enables programming at the scope of an entire system board. A single interface accepts CPU control via CLIF functions, a shared memory arena, and a selection of primitive functions CLIF can call, which cover device, display and other board behavior. The artifact and driver API allow Base to be embedded both in systems code, which interacts with existing artifacts, and in proof assistants, which generate artifacts and invoke the driver through its C API. A Lean codebase allows programming applications at native speed while embedding proofs about their behavior.

References

[1] Bytecode Alliance. *Cranelift: A Code Generation Toolkit*. https://cranelift.dev/.

[2] Chris Lattner and Vikram Adve. *LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation*. CGO, 2004.

[3] NVIDIA. *CUDA C++ Programming Guide*. https://docs.nvidia.com/cuda/.

[4] Carsten Bormann and Paul Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949, IETF, 2020.

[5] Nicholas D. Matsakis and Felix S. Klock II. *The Rust Language*. HILT, 2014.

[6] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.

[7] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE, 2021.